

Национальный исследовательский университет "Высшая школа экономики"
Факультет компьютерных наук
Образовательная программа "Программная инженерия"

СОГЛАСОВАНО

Старший разработчик в X5 Tech

_____ В. Е. Козловский

« ____ » _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
"Программная инженерия", старший
преподаватель департамента
программной инженерии

_____ Н. А. Павлочев

« ____ » _____ 2026 г.

**РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОГО АССИСТЕНТА ДЛЯ
ОБРАЗОВАТЕЛЬНЫХ УЧРЕЖДЕНИЙ**

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.05.09 81 01-1-ЛУ

Исполнители:

Студент группы БПИ236

_____ / С. Е. Растворов /

« ____ » _____ 2026 г.

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.05.09 81 01-1-ЛУ

**РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОГО АССИСТЕНТА ДЛЯ
ОБРАЗОВАТЕЛЬНЫХ УЧРЕЖДЕНИЙ**

Пояснительная записка

RU.17701729.05.09 81 01-1

Листов 16

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Пояснительная записка содержит сведения о назначении программы, области ее применения, технических характеристиках, выбранных средствах реализации, архитектурных решениях и ожидаемых технико-экономических показателях разработки.

Документ подготовлен в составе программной документации курсового проекта «Разработка интеллектуального ассистента для образовательных учреждений» в соответствии с требованиями Единой системы программной документации.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	4
1.1. Наименование программы	4
1.2. Документ(ы), на основании которого(ых) ведется разработка	4
2. НАЗНАЧЕНИЕ РАЗРАБОТКИ	5
2.1. Функциональное назначение	5
2.2. Эксплуатационное назначение	5
2.3. Краткая характеристика области применения	5
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	6
3.1. Постановка задачи на разработку программы	6
3.2. Результаты экспериментального сравнения языковых моделей	6
3.3. Описание и обоснование выбора состава технических и программных средств	6
3.3.1. Состав технических и программных средств	6
3.3.2. Обоснование выбора технических и программных средств	7
3.4. Описание и обоснование архитектуры программы	7
3.4.1. Границы безопасности	9
3.5. Особенности функционала	9
3.5.1. Основной сценарий работы	9
3.5.2. Резервный ответ при недоступности языковой модели	9
3.5.3. Загрузка и индексация документов	9
3.5.4. История диалогов	10
3.5.5. Контур развёртывания	10
3.5.6. Наблюдаемость	10
3.6. Обоснование выбора метода организации входных и выходных данных	11
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	12
4.1. Ориентировочная экономическая эффективность	12
4.2. Предполагаемая потребность	12
4.3. Сравнение с другими подходами	12
ОПИСАНИЕ ПРИМЕНЕНИЯ ГЕНЕРАТИВНОЙ МОДЕЛИ	13
ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ	14
ПРИЛОЖЕНИЕ 2. ТЕРМИНОЛОГИЯ	15

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ**1.1. Наименование программы**

Наименование программы – «Разработка интеллектуального ассистента для образовательных учреждений».

Наименование программы на английском языке – «Development of an Intelligent Assistant for Educational Institutions».

1.2. Документ(ы), на основании которого(ых) ведется разработка

Разработка ведется на основании учебного плана подготовки бакалавров по направлению 09.03.04 «Программная инженерия» и утвержденной академическим руководителем программы темы курсового проекта.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ РАЗРАБОТКИ

2.1. Функциональное назначение

Программа «Разработка интеллектуального ассистента для образовательных учреждений» предназначена для подготовки ответов на вопросы студентов и сотрудников образовательного учреждения по внутренним документам. Для формирования ответа применяется подход RAG (Retrieval-Augmented Generation): программа выполняет поиск фрагментов документов, относящихся к вопросу, передаёт найденный контекст языковой модели и возвращает пользователю ответ вместе со сведениями об источниках.

Функциональное назначение включает следующие задачи: семантический поиск фрагментов внутренних документов по вопросу пользователя; генерация ответа со сведениями об использованных источниках; формирование резервного ответа при недоступности языковой модели; хранение ограниченного контекста диалогов веб- и Telegram-пользователей; индексация корпуса отдельной командой и начальная индексация пустого хранилища при запуске сервера; сбор технических счётчиков и данных о длительности обработки; доступ к API по ключу; аутентификация веб-пользователей и разграничение административных операций по ролям.

2.2. Эксплуатационное назначение

Программа предназначена для эксплуатации в составе контейнеров Docker Compose, разворачиваемых на сервере учебного учреждения. В состав программных средств входят Ollama с моделью Mistral для генерации ответов, ChromaDB с сохраняемым на диске векторным индексом, SQLite для учётных записей и сессий веб-пользователей и ограниченного контекста диалогов, а также PostgreSQL 16 для данных пользователей Telegram и журнала их обращений. Штатными пользовательскими интерфейсами являются основной веб-интерфейс и Telegram-бот. Интерфейс Streamlit используется как вспомогательное средство проверки API.

Целевые эксплуатационные характеристики, установленные техническим заданием, предусматривают пилотную группу из 50—100 активных пользователей, до 300 запросов в сутки и доступность не менее 95% в рабочее время 09:00—21:00 МСК. Соответствие этим значениям подлежит проверке в ходе испытаний. Специальной подготовки рабочего места пользователя не требуется: для работы через Telegram необходим стандартный клиент мессенджера, для работы через веб — современный браузер.

2.3. Краткая характеристика области применения

Область применения программы — информационное обслуживание участников образовательного процесса. Система ориентирована на сценарии, в которых пользователь задаёт конкретный вопрос по документам вуза и получает ответ с указанием источника, не прибегая к ручному поиску по PDF-файлам или веб-страницам.

С точки зрения состава пользователей система охватывает три группы: студентов бакалавриата и магистратуры (учебные планы, сроки, процедуры), преподавателей и учебных офисов (регламенты, сервисы), ИТ-служб и методистов (быстрые справки). Программа дополняет официальные FAQ-разделы сайтов вузов, однако, в отличие от них, опирается на внутренние документы конкретного учреждения и даёт ответы с точными ссылками на источники.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку программы

Программа «Разработка интеллектуального ассистента для образовательных учреждений» предназначена для подготовки ответов на вопросы студентов и сотрудников образовательного учреждения по внутренним документам. Для формирования ответа применяется подход RAG (Retrieval-Augmented Generation): программа выполняет поиск фрагментов документов, относящихся к вопросу, передаёт найденный контекст языковой модели и возвращает пользователю ответ вместе со сведениями об источниках.

Программа предоставляет интерфейс прикладного программирования (API), основной веб-интерфейс и Telegram-бот. Серверная часть должна обеспечивать проверку входных данных, разграничение доступа к API и веб-интерфейсу, хранение ограниченного контекста диалога для веб- и Telegram-пользователей, обработку ошибок зависимых компонентов и сбор технических метрик. Генерация выполняется моделью Mistral, запущенной локально средствами Ollama, поэтому содержание документов не требуется передавать внешнему поставщику модели.

При недоступности языковой модели программа формирует резервный ответ только на основании фрагментов, уже найденных в векторном хранилище. Показатели качества ответов, времени обработки запросов и доступности программы относятся к целевым характеристикам и подлежат проверке по программе и методике испытаний.

3.2. Результаты экспериментального сравнения языковых моделей

В индивидуальной программе и методике испытаний А. К. Соколовой приведены результаты сравнения восьми языковых моделей по ответам на 14 контрольных вопросов. Наибольшее значение итогового показателя Q составило 2,64 из 3 для модели qwen2.5:3b; средняя оценка точности в материалах испытаний составила 2,43 из 3. Среднее время формирования ответа для разных моделей находилось в пределах от 15,30 до 101,69 с, для модели mistral:7b — 86,19 с.

3.3. Описание и обоснование выбора состава технических и программных средств

3.3.1. Состав технических и программных средств

Серверная часть программы реализована на языке Python 3.12 и развёртывается с помощью Docker Compose. В состав программных средств входят:

- FastAPI — HTTP-фреймворк серверной части;
- LangChain — библиотека для формирования запросов к языковой модели и интеграции компонентов RAG;
- ChromaDB — векторное хранилище фрагментов документов и их метаданных;
- Hugging Face Transformers и модель cointegrated/rubert-tiny2 — средства вычисления векторных представлений текста;
- Ollama и модель Mistral — средства локальной генерации ответов;
- SQLAlchemy — слой доступа к реляционным данным;
- SQLite и aiosqlite — используемое по умолчанию хранилище учётных записей, веб-сессий и ограниченного контекста диалогов;
- PostgreSQL 16 и asyncpg — хранилище пользователей Telegram-бота и журнала их обращений;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

- `pypdf` — извлечение текстового содержимого PDF-файлов;
- `Tesseract OCR` и `Pillow` — распознавание текста на изображениях, переданных Telegram-боту;
- `aiogram 3.x` — библиотека Telegram-бота;
- `Jinja2`, `HTML`, `CSS` и `JavaScript` — основной веб-интерфейс;
- `Streamlit` — вспомогательный интерфейс для проверки серверной части;
- `Prometheus client` — формирование технических метрик;
- `Docker` и `Docker Compose` — контейнеризация и согласованный запуск компонентов.

Для управления зависимостями используется `uv`, для статической проверки и форматирования исходного кода — `ruff`, для автоматизированного тестирования — `pytest`.

3.3.2. Обоснование выбора технических и программных средств

`FastAPI` выбран как основа серверной части благодаря поддержке асинхронных обработчиков, механизма зависимостей и автоматического формирования схемы `OpenAPI`. Эти возможности позволяют отделить проверку доступа и входных данных от обработки вопроса и использовать единый сервер для API и веб-интерфейса.

`LangChain` предоставляет совместимые компоненты для работы с шаблоном запроса к языковой модели, векторным хранилищем и моделями векторного представления текста. `ChromaDB` поддерживает хранение векторного индекса на диске без отдельного серверного процесса. Модель `cointegrated/rubert-tiny2` используется для обработки русскоязычного текста при умеренных требованиях к вычислительным ресурсам.

`Ollama` обеспечивает локальный запуск модели `Mistral`. После предварительной загрузки моделей штатная генерация ответов не требует обращения к внешнему API, что сокращает число внешних зависимостей и позволяет обрабатывать внутренние документы в локальном контуре.

`SQLAlchemy` используется в обеих подсистемах реляционного хранения. Веб-подсистема по умолчанию использует `SQLite`, что упрощает запуск пилотного стенда; строка подключения задаётся параметром `WEB_AUTH_DATABASE_URL` и может быть изменена без переработки моделей данных. Telegram-бот использует `PostgreSQL` через параметр `DATABASE_URL` для долговременного хранения пользователей, исходных обращений и сформированных ответов.

Библиотеки `pypdf` и `Tesseract` решают разные задачи. `pypdf` извлекает доступный текст из PDF-документов, а `Tesseract` распознаёт текст на изображениях, полученных Telegram-ботом. `Aiogram` обеспечивает асинхронную обработку сообщений Telegram. `Docker Compose` задаёт состав контейнеров, их зависимости, переменные окружения, тома и проверки состояния.

3.4. Описание и обоснование архитектуры программы

Центральным компонентом программы является RAG-сервер на `FastAPI`. Он предоставляет методы API, обслуживает основной веб-интерфейс, обращается к `ChromaDB` и `Ollama`, а также использует отдельную базу данных веб-аутентификации. Telegram-бот и вспомогательный `Streamlit`-клиент обращаются к серверу по HTTP. Такое разделение позволяет изменять клиентские интерфейсы без изменения последовательности обработки вопросов.

Для обработки вопросов предусмотрены два метода API:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

- POST /ask требует заголовок X-API-Key; обычные вызовы API выполняются без контекста, а доверенные вызовы Telegram-бота связываются с контекстом конкретного Telegram-пользователя;
- POST /web/ask допускает доступ по ключу API либо по действующей серверной веб-сессии; контекст предыдущих вопросов загружается только для аутентифицированного веб-пользователя.

Оба метода возвращают объект JSON с полями `answer`, `sources` и `metadata`. Поле `sources` содержит найденные фрагменты и сведения об исходных документах. В поле `metadata` передаются модель, число источников, расчётный показатель уверенности, признак и причина резервного ответа, а также длительность этапов обработки.

Обработка вопроса выполняется в следующей последовательности:

1. сервер проверяет авторизацию, формат и длину вопроса;
2. для аутентифицированного веб- или Telegram-пользователя загружаются последние сохранённые вопросы;
3. из текущего вопроса и ограниченного контекста формируется поисковый запрос;
4. в ChromaDB выполняется поиск k наиболее близких фрагментов;
5. если фрагменты не найдены, возвращается сообщение об отсутствии информации в корпусе;
6. если фрагменты найдены, вопрос, контекст диалога и найденные фрагменты передаются локальной языковой модели;
7. при ошибке модели, превышении времени ожидания либо исчерпании общего временного бюджета формируется резервный ответ из найденных фрагментов;
8. сервер добавляет сведения об источниках и метаданные, регистрирует метрики и возвращает результат клиенту.

Пустой векторный индекс и недоступность ChromaDB обрабатываются отдельно и приводят к ответу API с кодом 503. Эти состояния не считаются резервным ответом языковой модели.

Основной веб-интерфейс формируется сервером из шаблона `src/server/templates/index.html` и отправляет вопросы в POST /web/ask. Telegram-бот из каталога `src/bot` преобразует текстовое сообщение или извлечённый из вложения текст в вопрос и обращается к POST /ask. Streamlit-клиент из файла `src/client/app/streamlit_app.py` используется как вспомогательное средство проверки API.

Подготовка корпуса разделена на извлечение текста, разбиение документов на фрагменты и сохранение их в ChromaDB. Модуль `document_ingestion.py` обрабатывает файлы PDF, HTML, HTM, TXT и DOCX. Размер фрагмента и перекрытие задаются параметрами `RAG_CHUNK_SIZE` и `RAG_CHUNK_OVERLAP`; значения по умолчанию составляют 500 и 100 символов соответственно.

Индексация запускается отдельной командой модуля `src.server.app.index_documents`. Кроме того, при запуске сервера может быть выполнена начальная индексация, если подготовка RAG включена, а векторное хранилище не содержит документов. Эти режимы управляются параметрами `PREPARE_RAG_ON_STARTUP` и `AUTO_INDEX_ON_STARTUP`. Отдельный HTTP-метод для загрузки или фонового обновления индекса в текущей версии отсутствует.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3.4.1. Границы безопасности

Доступ к POST /ask и GET /metrics проверяется по ключу API. Основной веб-интерфейс использует серверные учётные записи и сессионные файлы cookie с признаками HttpOnly и SameSite=Lax; административные операции дополнительно проверяют роль пользователя. Для методов обработки вопросов установлено ограничение частоты обращений, а длина вопроса и идентификатора клиентской сессии ограничена до передачи данных в RAG.

Перечисленные меры контролируют доступ и ограничивают потребление ресурсов, но не обеспечивают защиту от внедрения инструкций в запрос к модели (prompt injection). История вопросов, найденные фрагменты и текущий вопрос помещаются в общий шаблон запроса. Поэтому прямое внедрение инструкций через вопрос, косвенное внедрение через содержимое корпуса, накопление вредоносного контекста и подмена материалов корпуса остаются открытыми рисками текущей архитектуры.

Языковой модели не предоставлены средства вызова прикладных функций, выполнения команд операционной системы или прямого изменения базы данных. Это ограничивает возможные последствия внедрения инструкций, однако не предотвращает формирование ложного ответа, игнорирование правил шаблона или раскрытие доступного модели контекста. Поэтому текущая версия программы не рассматривается как защищённая от атак на инструкции языковой модели.

3.5. Особенности функционала

3.5.1. Основной сценарий работы

Пользователь задаёт вопрос через Telegram-бот или основной веб-интерфейс. Telegram-бот передаёт вопрос в POST /ask, а веб-интерфейс — в POST /web/ask. Сервер выполняет поиск фрагментов в ChromaDB, формирует ответ с помощью локальной языковой модели и возвращает текст ответа, перечень источников и технические метаданные. Клиентский интерфейс преобразует полученный JSON в сообщение, предназначенное для пользователя.

3.5.2. Резервный ответ при недоступности языковой модели

Резервный ответ формируется только после успешного поиска непустого набора фрагментов. Если вызов Ollama завершается ошибкой, превышает установленное время ожидания либо общий временной бюджет RAG исчерпан до начала генерации, сервер возвращает наиболее релевантные найденные фрагменты. Метаданные такого ответа содержат fallback_used: true и значение fallback_reason, указывающее причину перехода в резервный режим.

Резервный режим не выполняет повторный лексический поиск и не использует отдельный модуль поиска. При отсутствии найденных фрагментов возвращается сообщение о недостатке информации с признаком fallback_used: false.

3.5.3. Загрузка и индексация документов

Оператор размещает документы корпуса в каталоге, заданном параметром DOCUMENTS_DIR, после чего запускает модуль индексации из командной строки. Поддерживаются файлы .pdf, .html, .htm, .txt и .docx. Для каждого документа извлекаются текст и метаданные, затем текст разбивается на перекрывающиеся фрагменты, для которых вычисляются векторные

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

представления. Полученные данные сохраняются в каталоге ChromaDB, заданном параметром VECTOR_DB_DIR.

Изображения не входят в перечень форматов корпуса. Фотографии, переданные Telegram-боту, распознаются Tesseract и преобразуются в текст пользовательского вопроса. Из PDF-вложений Telegram извлекается доступный текст с помощью rupdf; распознавание отсканированных PDF-страниц в этой последовательности не выполняется.

3.5.4. История диалогов

Для аутентифицированного веб- или Telegram-пользователя сервер сохраняет тексты его предыдущих вопросов. Веб-записи связываются с серверным ключом вида web:<идентификатор пользователя>, а Telegram-записи — с ключом вида tg:<идентификатор Telegram-пользователя>. Оба ключа формируются сервером и не принимаются как произвольный клиентский идентификатор. Записи хранятся в базе, заданной параметром WEB_AUTH_DATABASE_URL. Число последних вопросов, срок хранения и предельное число ключей регулируются параметрами CONVERSATION_MEMORY_WINDOW, CONVERSATION_MEMORY_TTL_SECONDS и CONVERSATION_MEMORY_MAX_SESSIONS.

Произвольный клиентский session_id не используется для выбора контекста. При вызове из Telegram сервер связывает идентификатор пользователя с доверенными учётными данными Telegram-бота и сам формирует ключ памяти. Вызов POST /web/ask только с ключом API, но без аутентифицированного веб-пользователя, выполняется без контекста.

Telegram-бот дополнительно сохраняет исходное обращение и сформированный ответ в PostgreSQL. Этот журнал предназначен для учёта обращений; ограниченный RAG-контекст управляется сервером отдельно от этого журнала.

3.5.5. Контур развёртывания

Локальный состав контейнеров описан в файле docker-compose.yaml и запускается с профилем dev. Состав промышленного контура описан в файле deployment/production/compose.yaml. В обоих случаях сервер, Telegram-бот и реляционная база запускаются как отдельные сервисы; каталог документов и данные ChromaDB подключаются к серверу как тома.

Основные параметры задаются переменными окружения. К ним относятся API_KEY, WEB_BOOTSTRAP_ADMIN_TOKEN, WEB_AUTH_DATABASE_URL, BOT_TOKEN, DATABASE_URL, OLLAMA_HOST, LLM_MODEL, HF_EMBEDDING_MODEL, VECTOR_DB_DIR, DOCUMENTS_DIR, RAG_TOP_K, ограничения времени обработки и параметры памяти диалога. Параметры PREPARE_RAG_ON_STARTUP и AUTO_INDEX_ON_STARTUP управляют подготовкой индекса при запуске.

Метод GET /health используется контейнерной системой для проверки доступности процесса FastAPI. Он возвращает состояние самого HTTP-сервера и не выполняет углублённую проверку Ollama, ChromaDB или реляционной базы данных.

3.5.6. Наблюдаемость

Сервер публикует метрики в формате Prometheus через защищённый ключом API метод GET /metrics. Регистрируются следующие показатели:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

- rag_requests_total — число обращений к методам обработки вопросов;
- rag_fallback_total — число сформированных резервных ответов;
- rag_errors_total — число ошибок с разделением по этапу обработки;
- rag_retrieval_seconds — распределение длительности поиска документов;
- rag_generation_seconds — распределение длительности генерации ответа;
- rag_total_seconds — распределение полной длительности обработки вопроса.

3.6. Обоснование выбора метода организации входных и выходных данных

Входные данные программы разделены на пользовательские вопросы, документы корпуса и параметры конфигурации. Текстовые вопросы поступают в формате JSON через методы API. Основной веб-интерфейс и Telegram-бот самостоятельно формируют соответствующие HTTP-запросы. Изображения и PDF-файлы, полученные Telegram-ботом, предварительно преобразуются в текст и используются как пользовательский вопрос, не попадая документы корпуса.

Документы корпуса поступают через каталог файлов. Для индексации допускаются форматы PDF, HTML, HTM, TXT и DOCX. Параметры подключения к зависимым компонентам, ограничения времени, размеры фрагментов и режим начальной индексации задаются переменными окружения. Такое разделение не смешивает пользовательский ввод с управляемым оператором корпусом документов.

Результат обработки вопроса возвращается в формате JSON. Поле `answer` содержит текст ответа, `sources` — список использованных фрагментов и их метаданных, `metadata` — технические сведения о выполнении запроса. Показатель `confidence` и признаки `fallback_used` и `fallback_reason` являются вложенными полями объекта `metadata`.

Внутреннее хранение также разделено по назначению. Векторные представления, тексты фрагментов и их метаданные хранятся в ChromaDB. Данные веб-пользователей, веб-сессий и ограниченного контекста веб- и Telegram-диалогов хранятся в базе WEB_AUTH_DATABASE_URL, по умолчанию SQLite. Данные пользователей Telegram и полный журнал их обращений сохраняются в PostgreSQL. Ключ API и остальные параметры конфигурации передаются через переменные окружения и не относятся к пользовательской истории.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Ориентировочная экономическая эффективность

Разработка программы «Разработка интеллектуального ассистента для образовательных учреждений» не ориентирована на получение прямой коммерческой выручки. Предполагаемый эффект связан с возможным снижением трудоёмкости информационного обслуживания студентов и преподавателей за счёт автоматизации поиска сведений во внутренних документах и подготовки ответов на типовые вопросы.

Ожидаемый эффект может выражаться в сокращении времени поиска и подготовки типового ответа, уменьшении необходимости повторно формулировать контекст предыдущего вопроса, снижении зависимости от внешних API языковых моделей и возможности применять программные компоненты с разными управляемыми корпусами документов. Количественная оценка сокращения времени, числа обращений и эксплуатационных расходов не проводилась.

4.2. Предполагаемая потребность

Потребность в программе связана с повторяющимися вопросами студентов по регламентам, срокам и процедурам. В соответствии с техническим заданием планируемый объём пилотной эксплуатации составляет 50—100 активных пользователей и до 300 запросов в сутки.

Предполагаемыми пользователями являются студенты бакалавриата и магистратуры, преподаватели, сотрудники учебных офисов. Для них программа может служить средством получения справок по учебным планам, регламентам и процедурам

4.3. Сравнение с другими подходами

Статический справочный раздел (часто задаваемые вопросы) содержит заранее подготовленные ответы и требует ручного обновления материалов. Во внешнем универсальном языковом сервисе размещение модели, правила обработки данных и доступность определяются поставщиком. Разработанная программа использует управляемый локальный корпус, локально запущенную модель, возвращает сведения об источниках и поддерживает ограниченный контекст веб- и Telegram-диалога. При недоступности модели после успешного векторного поиска формируется резервный ответ.

Это сравнение отражает архитектурные различия, конкретное экономическое превосходство не оценивалось. Сервис был предоставлен на тестирование представителем целевой аудитории и отзывы носили позитивный сантмент.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ОПИСАНИЕ ПРИМЕНЕНИЯ ГЕНЕРАТИВНОЙ МОДЕЛИ

При подготовке данной курсовой работы генеративные модели использовались как вспомогательный инструмент при работе с текстовой документацией. Использование не носило автономного характера: итоговые формулировки, отбор фактов, проверка соответствия техническому заданию, сопоставление с исходным кодом и окончательная редактура выполнялись авторами работы.

Генеративная модель применялась для следующих целей:

- подготовки черновых формулировок отдельных фрагментов пояснительной записки, программы и методики испытаний, руководства оператора и связанной программной документации;
- унификации терминологии и стиля изложения в разных документах комплекта;
- проверки согласованности между текстом пояснительной записки, техническим заданием и фактическим составом функций проекта;
- редактирования русскоязычных технических формулировок и сокращения числа стилистических несоответствий;
- подготовки черновых вариантов отдельных фрагментов программного кода, конфигурации и служебных сценариев при последующей ручной доработке.

В работе использовалась генеративная модель OpenAI Codex, доступная через инструментальную среду Codex. Адрес источника в сети «Интернет»: <https://openai.com/>. Способ применения модели: формирование черновых вариантов текста, предложение альтернативных формулировок, редактура отдельных разделов, выявление потенциальных противоречий между документами, а также подготовка черновых фрагментов кода и конфигурации. Использование модели не означало автоматического принятия сгенерированного результата: все предложенные фрагменты программного кода, конфигурации и документации проходили ручную проверку, адаптацию к структуре проекта и последующее тестирование либо сверку с исходным кодом. Содержательные решения о составе системы, описании архитектуры, перечне функций и результатах испытаний принимались авторами работы на основании исходного кода, технического задания и сопутствующей документации проекта.

С точки зрения состава работы использование генеративной модели относится не ко всей работе целиком, а к отдельным фрагментам текста, программного кода и конфигурации. Наибольший объём применения пришёлся на разделы, связанные с выравниванием стиля, согласованием терминологии, подготовкой служебной документации и получением черновых вариантов технической реализации. Оценка эффективности использования модели является положительной: применение модели позволило ускорить подготовку черновых формулировок, техническую редактуру и предварительную кодогенерацию, однако не заменяло ручную проверку фактов, сверку с кодовой базой, контроль соответствия ТЗ и окончательное авторское редактирование.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
14. Правила использования искусственного интеллекта студентами НИУ ВШЭ. URL: https://www.hse.ru/studyspravka/ai_guidelines/ (дата обращения: 11.05.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2. ТЕРМИНОЛОГИЯ

Термин	Определение
RAG	Retrieval-Augmented Generation — подход, при котором перед генерацией ответа языковая модель получает сведения, найденные во внешнем по отношению к ней источнике знаний.
ChromaDB	Векторное хранилище данных, предназначенное для сохранения векторных представлений, метаданных и поиска по сходству.
Эмбединг	Числовое векторное представление объекта, сохраняющее признаки, необходимые для сравнения по смысловой близости.
Чанк	Фрагмент данных, полученный при разделении исходного документа на части для последующей обработки.
LLM	Large Language Model — большая языковая модель, обученная обрабатывать и формировать текст.
Fallback	Резервный способ обработки, применяемый при недоступности или ошибке основного способа.
OCR	Optical Character Recognition — оптическое распознавание символов, то есть преобразование текста на изображении в машиночитаемый вид.
Confidence	Числовой показатель степени соответствия результата заданному критерию или мере сходства. В данной программе он рассчитывается по расстояниям до найденных фрагментов и не является вероятностью правильности ответа или самооценкой языковой модели.
REST/JSON API	Прикладной программный интерфейс, использующий архитектурный стиль REST, протокол HTTP и формат JSON для обмена данными.
Ролевая модель	Способ разграничения доступа, при котором разрешённые операции определяются назначенной пользователю ролью.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 81 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]